



APLICACIONES MÓVILES

DOCENTE: GABRIEL EDUARDO MOREJÓN
LÓPEZ

ACTIVIDAD 1: PRÁCTICA 1

UNIDAD 1: ARQUITECTURA DE APLICACIONES MÓVILES

NOMBRES Y APELLIDOS DEL ESTUDIANTE

MICHAEL AGUSTIN PALMA NAVARRETE

LEIBER VALENTIN ZAMBRANO CHAVEZ

BLAIR STEVEN DELGADO CORNEJO

CHRISTOFER JESUS BARZOLA INDARTE

25 DE MAY DE 2026

UNIDAD 1: ARQUITECTURA DE APLICACIONES MÓVILES

GitHub Repositorio: https://github.com/RTgo14/Sistema_HelpDesk

zip del Proyecto:

<https://drive.google.com/file/d/1duoFzx1GSejan1RxxasQpiT8Ql0kf5YO/view?usp=sharing>

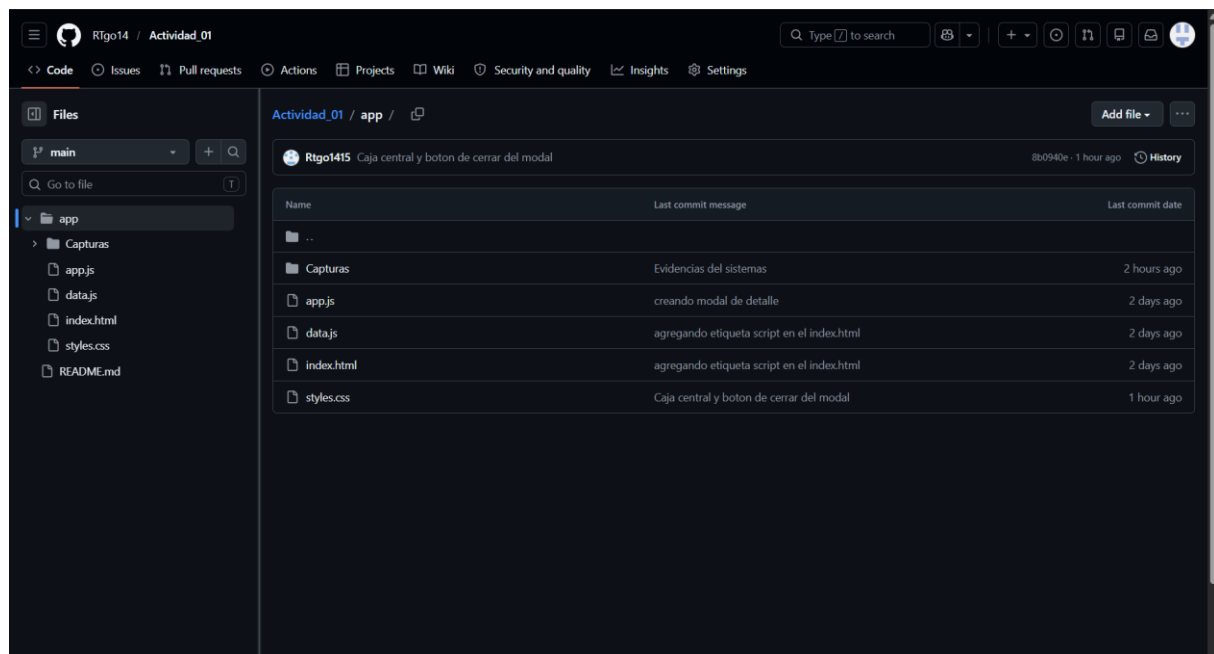
1. INTRODUCCIÓN

Este proyecto consiste en el desarrollo de un Dashboard Web Interactivo destinado a una Mesa de Ayuda e Incidencias Tecnológicas, funciona como una interfaz centralizada y dinámica para listar, visualizar y administrar de forma estructurada reportes de soporte técnico dentro de una organización. Esta práctica se realiza con el fin de fortalecer competencias en el desarrollo front-end nativo con JS intermedio sin uso de frameworks, teniendo de propósito optimizar la gestión operativa mediante búsquedas predictivas, filtros avanzados, validación estricta de formularios y persistencia de estados. Para el desarrollo del programa usamos control de versiones en GitHub, organizando el espacio de trabajo en los archivos esenciales index.html, styles.css, data.js con datos semilla estructurados en arreglos de objetos y app.js. Para hacer el sistema dinámico usamos JS ES6 empleando métodos de orden superior (map, filter, find) para la reactividad de los datos [1], captura de eventos del DOM para el control del formulario y el despliegue del modal de detalle, junto a la integración de localStorage para garantizar la persistencia local de la información [2].

2. DESARROLLO

1. Planificación Inicial

Se definió el módulo denominado **Mesa de Ayuda e Incidencias Tecnológicas**, estableciendo como entidad principal la "**Incidencia**". Se determinaron los campos mínimos estructurales para el control operativo: identificador único (id), título, descripción, prioridad, estado y fecha. Con esto claro, se inicializó el repositorio remoto en GitHub configurando un espacio de trabajo modular.



2. Construcción de la Estructura HTML

Se diseñó el esqueleto semántico de la aplicación en el archivo index.html. La interfaz se dividió mediante un contenedor principal (<main class="container">) estructurado en dos secciones críticas: un panel izquierdo con un formulario (#form-incidencia) equipado con campos de texto y selectores para la captura de datos, y un panel derecho (.list-section) enfocado en la visualización, provisto de una barra de filtros interactivos y una cuadrícula contenedora (#lista-incidencias). Adicionalmente, se integró un bloque fuera de flujo para la ventana modal (#modal-detalle).

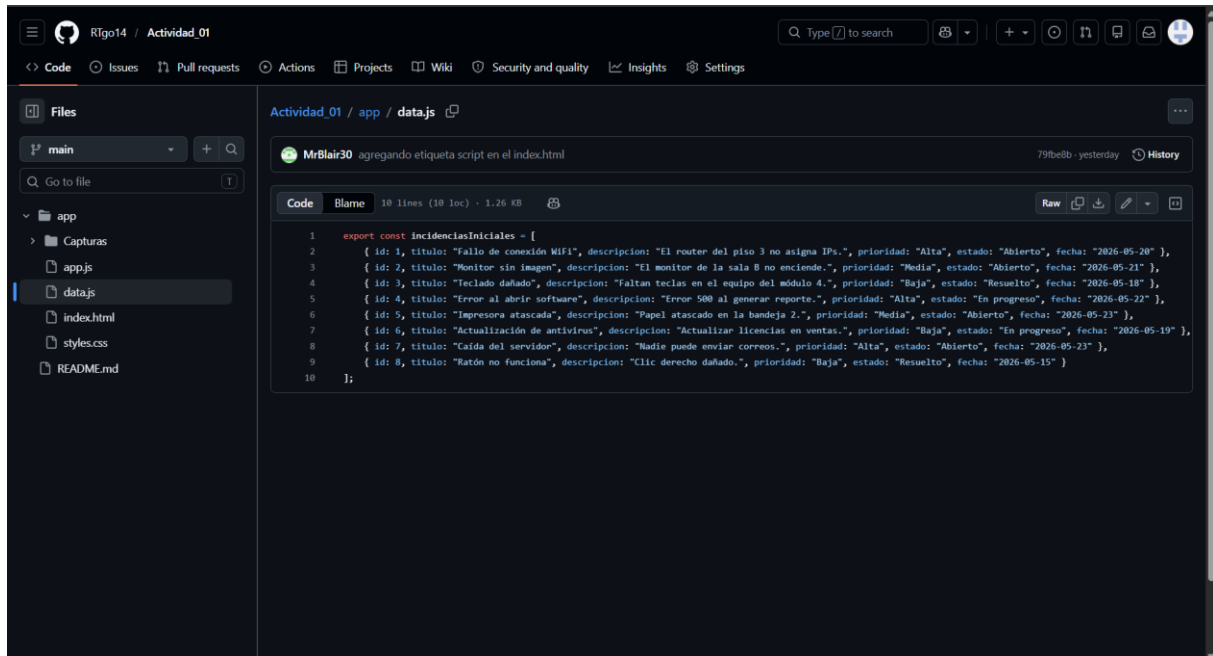
```
<main class="container">
  <!-- Panel Izquierdo: Formulario -->
  <section class="form-section">
    <h2>Registrar Incidencia</h2>
    <form id="form-incidencia">
      <div class="form-group">
        <label for="titulo">Título de la incidencia:</label>
        <input type="text" id="titulo" placeholder="Ej. Fallo en servidor">
      </div>
      <div class="form-group">
        <label for="descripcion">Descripción detallada:</label>
        <textarea id="descripcion" rows="4" placeholder="Describe el problema..."></textarea>
      </div>
      <div class="form-group">
        <label for="prioridad">Prioridad:</label>
        <select id="prioridad">
          <option value="">Selecione...</option>
          <option value="Alta">Alta</option>
          <option value="Media">Media</option>
          <option value="Baja">Baja</option>
        </select>
      </div>
      <!-- Mensajes de validación -->
      <div id="mensaje-error" class="error-msg hidden"></div>
      <button type="submit" class="btn-primary">Guardar Incidencia</button>
    </form>
  </section>
```

```
<!-- Panel Derecho: Formulario -->
<section class="list-section">
  <div class="filters">
    <input type="text" id="buscar" placeholder="Buscar por título...">
    <select id="filtro-prioridad">
      <option value="Todas">Todas las prioridades</option>
      <option value="Alta">Alta</option>
      <option value="Media">Media</option>
      <option value="Baja">Baja</option>
    </select>
  </div>

  <div id="lista-incidencias" class="incidencias-grid">
    <!-- Se renderiza dinámicamente con JS -->
  </div>
</section>
</main>
```

3. Carga de Datos de Ejemplo

Para garantizar la consistencia inicial del panel y permitir pruebas de volumen controlado, se creó el archivo data.js. En él se declaró y exportó un arreglo constante llamado incidenciasIniciales, compuesto exactamente por 8 objetos de JavaScript estructurados con datos realistas de fallos tecnológicos cotidianos.



4. Renderizado Dinámico

La inyección dinámica de datos en el DOM se centralizó en la función renderizarIncidencias dentro de app.js. El flujo procesa el arreglo mediante una copia controlada aplicando el método de orden superior .sort() para ordenar los registros de forma descendente por su id. Posteriormente, implementa el método .map() [3].

```
// Renderizado Dinamico
const renderizarIncidencias = (datos) => {
  listaDOM.innerHTML = "";

  if (datos.length === 0) {
    listaDOM.innerHTML =
      '<p class="empty-msg">No se encontraron incidencias.</p>';
    return;
  }

  // METODO sort
  const datosOrdenados = [...datos].sort((a, b) => b.id - a.id);

  // METODO map
  const htmlString = datosOrdenados
    .map(
      (incidencia) => `
    <div class="card card-${incidencia.prioridad.toLowerCase()}">
      <h3>${incidencia.titulo}</h3>
      <p class="badge badge-${incidencia.prioridad.toLowerCase()}">Prioridad: ${incidencia.prioridad}</p>
      <p><strong>Estado:</strong> ${incidencia.estado}</p>
      <p><small>${incidencia.fecha}</small></p>
      <button class="btn-detalle" data-id="${incidencia.id}">Ver Detalle</button>
    </div>
  `
    )
    .join("");

  listaDOM.innerHTML = htmlString;
  asignarEventosDetalle();
};
```

5. Implementación de Búsqueda y Filtros

Se desarrolló una lógica de filtrado reactiva integrada en la función `aplicarFiltros`. Esta función se dispara mediante escuchadores de eventos (`addEventListener`) asociados a los tipos de interacción `input` y `change` [4]. El algoritmo limpia las cadenas de caracteres mediante `toLowerCase()` y ejecuta el método de arreglos `.filter()`, evaluando en paralelo si el título incluye el texto buscado (`.includes()`) y si la prioridad seleccionada coincide con los registros [3].

```
// Búsqueda y Filtros
const aplicarFiltros = () => {
  const textoBusqueda = inputBuscar.value.toLowerCase();
  const prioridadFiltro = selectFiltro.value;

  // METODO filter
  const resultados = incidencias.filter((inc) => {
    const coincideTexto = inc.titulo.toLowerCase().includes(textoBusqueda);
    const coincidePrioridad =
      prioridadFiltro === "Todas" || inc.prioridad === prioridadFiltro;
    return coincideTexto && coincidePrioridad;
  });

  renderizarIncidencias(resultados);
};
```

6. Formulario de Alta o Creación

Se configuró un manejador de eventos adjunto al formulario de registro interceptando el evento estándar de envío a través de `e.preventDefault()`, mecanismo proporcionado por JavaScript para controlar el comportamiento predeterminado de los eventos [3]. Al ejecutarse de forma válida, el controlador captura los valores de los campos limpiando espacios residuales con `.trim()`, genera un identificador temporal preciso basado en tiempo Unix mediante `Date.now()` y compone una nueva entidad estructural con el estado inicial prefijado en "Abierto", anexándola al arreglo de estados mediante el método reactivo `.push()`.

```
form.addEventListener('submit', (e) => {
  e.preventDefault();

  const titulo = document.getElementById('titulo').value.trim();
  const descripcion = document.getElementById('descripcion').value.trim();
  const prioridad = document.getElementById('prioridad').value;

  // Validacion
  if (!titulo || !descripcion || !prioridad) {
    mostrarError("Todos los campos son obligatorios.");
    return;
  }

  if (titulo.length < 5) {
    mostrarError("El título debe tener al menos 5 caracteres.");
    return;
  }

  msgError.classList.add('hidden');
```

```

const nuevaIncidencia = {
  id: Date.now(),
  titulo,
  descripcion,
  prioridad,
  estado: "Abierto",
  fecha: new Date().toISOString().split('T')[0]
};

incidencias.push(nuevaIncidencia);
localStorage.setItem('incidenciasIT', JSON.stringify(incidencias));

form.reset();
aplicarFiltros();
});

```

7. Validaciones

Para salvaguardar la integridad de los datos en la persistencia, se implementaron reglas restrictivas previas a la inserción en el flujo de creación. Mediante condicionales lógicos (if), el sistema verifica la presencia obligatoria de todos los parámetros; asimismo, restringe la longitud del campo de texto principal validando que no sea inferior a 5 caracteres (`titulo.length < 5`). Ante cualquier anomalía, se invoca la rutina `mostrarError()`.

```

// validacion
if (!titulo || !descripcion || !prioridad) {
  mostrarError("Todos los campos son obligatorios.");
  return;
}

if (titulo.length < 5) {
  mostrarError("El título debe tener al menos 5 caracteres.");
  return;
}

```

8. Pruebas

Se ejecutaron planes de pruebas funcionales en entorno de navegador web, verificando el correcto comportamiento interactivo del Dashboard.

Mesa de Ayuda - Incidencias Tecnológicas

Registrar Incidencia

Título de la incidencia:

Descripción detallada:

Prioridad:

Seleccione...
▼

Guardar Incidencia

Todas las prioridades ▼

Falla en el firewall de la red principal

Prioridad: Alta

Estado: Abierto

2026-05-25

Ver Detalle

Falla de tiempo de respuesta en el servidor

Prioridad: Alta

Estado: Abierto

2026-05-25

Ver Detalle

9. Documentación Mínima

Como fase de cierre del ciclo del desarrollo, se redactó el archivo README.md en la raíz del espacio de trabajo de GitHub.



3. BIBLIOGRAFÍA

- [1] “map, reduce y filter de JavaScript: Funciones de arreglo JS explicadas con ejemplos de código”, freeCodeCamp.org. Consultado: el 25 de mayo de 2026. [En línea]. Disponible en: <https://www.freecodecamp.org/espanol/news/map-reduce-y-filter-de-javascript-funciones-de-arreglo-js-explicadas-con-ejemplos-de-codigo/>
- [2] “Cómo usar LocalStorage en JavaScript”, freeCodeCamp.org. Consultado: el 25 de mayo de 2026. [En línea]. Disponible en: <https://www.freecodecamp.org/espanol/news/como-usar-localstorage-en-javascript/>
- [3] “JavaScript | MDN”, MDN Web Docs. Consultado: el 25 de mayo de 2026. [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Web/JavaScript>
- [4] “JavaScript DOM Events”. Consultado: el 25 de mayo de 2026. [En línea]. Disponible en: https://www.w3schools.com/JS/js_html_dom_events.asp